Sundial: A Family of Highly Capable Time Series Foundation Models

Yong Liu, Guo Qin, Zhiyuan Shi, Zhi Chen, Caiyin Yang, Xiangdong Huang, Jianmin Wang, Mingsheng Long

School of Software, BNRist, Tsinghua University

May 24, 2026

Background: Why probabilistic time-series forecasting matters

- We see time series drive decisions in energy, finance, logistics, healthcare, and weather.
- We need calibrated uncertainty, not just point forecasts.
- We aim to generalize across domains, handle long contexts, and respond fast.

The problem: Why we need native, scalable probabilistic forecasting

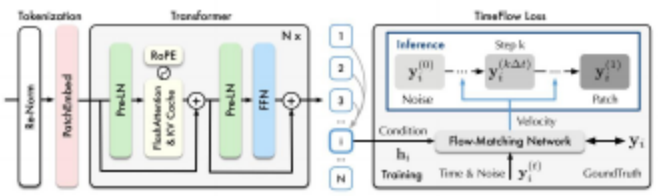
- We need native, scalable probabilistic forecasting for multimodal futures.
- We show parametric likelihoods restrict uncertainty and discrete tokenization harms context and calibration.
- We propose a native continuous, flexible, scalable generative approach.

We propose Sundial: core idea and contributions

- We introduce Sundial, a native continuous time-series foundation model that forecasts distributions.
- We propose TimeFlow Loss: conditional flow-matching for next-patch distributions without priors.
- We build an efficient decoder-only Transformer with continuous patch tokens and multi-patch prediction.
- We pre-train on a large corpus (TimeBench) and show strong zero-shot generalization.

Method overview: From continuous patches to generative forecasts

- We tokenize continuous patches and apply a causal decoder Transformer.
- We optimize TimeFlow Loss to learn next-patch distributions via flow-matching.
- At inference, we push forward from noise and sample multiple futures efficiently.



We summarize the architecture: patch tokenization + causal decoder + multi-patch prediction.

Data preparation and continuous patch tokenization

- We stationarize each instance (shift/outlier handling) and normalize per variable.
- We split series into length-P patches, pad and mask tails for arbitrary inputs.
- We embed values + masks with a shared MLP to D-dim tokens, shortening context vs. point-wise.

Backbone: Scalable, efficient decoder-only Transformer

- We use causal self-attention with Pre-LN and RoPE for stability and order awareness.
- We adopt FlashAttention for memory/throughput and KV cache for just-in-time generation.
- We predict multiple patches to reduce autoregressive steps in training and inference.

TimeFlow Loss: Intuition

- We learn conditional next-patch distributions without specifying a parametric prior.
- We flow-match velocities along paths from noise to target given context.
- We mitigate mode collapse and capture multimodal futures for calibrated uncertainty.

TimeFlow Loss: Training objective

- For token i, we condition the FM-Net on token representation h_i and time t.
- We interpolate $y_i^{(t)} = (1-t)\, y_i^{(0)} + t\, y_i$, where $t \sim \mathcal{U}[0,1]$, $y_i^{(0)} \sim \mathcal{N}(0, I)$.
- We regress the true velocity toward $y_i - y_i^{(0)}$ conditioned on h_i and t; same backbone as baselines.

TimeFlow Loss: Formal details

- We parameterize a conditional velocity field $v_\theta(y, t \mid h)$.
- We sample $t \sim \mathcal{U}[0,1]$, $y_0 \sim \mathcal{N}(0, I)$, and form $y(t) = (1-t)\, y_0 + t\, y$.
- We regress the true velocity toward $y - y_0$ conditioned on h and t.
- We avoid score estimation or diffusion SDEs; we use a direct flow-matching objective for next-patch distributions.

Sampling and complexity

- We integrate K steps with the learned velocity (push-forward) from Gaussian noise.
- We reuse the cached lookback representation and KV cache, so repeated samples are cheap.
- We scale with K and the number of predicted patches; multi-patch prediction reduces steps.

Inference: Fast probabilistic sampling

- We start from Gaussian noise and run K push-forward steps using the FM-Net and cached lookback.
- We resample with new noises to obtain many plausible futures and aggregate quantiles/median.
- We reuse the KV cache to enable millisecond zero-shot predictions and rapid resampling.

Experimental setup: Benchmarks and metrics

- We evaluate zero-shot on TSLib (point; MSE/MAE) and on FEV and GIFT-Eval (probabilistic; MASE/CRPS/WQL).

Key results: Zero-shot point forecasting on TSLib

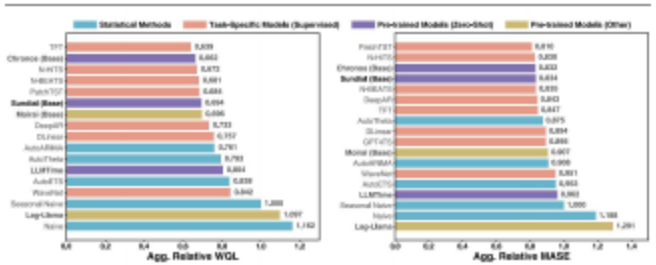
- We outperform prior SOTA Time-MoE with fewer parameters.
- We reduce avg MSE by 7.57% and MAE by 4.71% vs. Time-MoE.
- We find patch-level continuous tokens excel at long horizons.

We evaluate zero-shot on TSLib; lower MSE/MAE is better.

Models SundialSmall SundialBase SundialLarge Time-MoEBase Time-MoELarge Time-MoEUltra Timer-XL MoiraiBase MoiraiLarge ChronosBase ChronosLarge TimesFM (with respective references).
Metric MSE MAE for datasets: ETTm1, ETTm2, ETTh1, ETTh2, ECL, Weather; plus 1st Count.
ETTm1: 0.354 0.388 — 0.336 0.377 — 0.331 0.369 — 0.394 0.415 — 0.376 0.405 — 0.356 0.391 — 0.373 0.392 — 0.406 0.385 — 0.422 0.391 — 0.645 0.500 — 0.555 0.465 — 0.433 0.418
ETTm2: 0.265 0.324 — 0.258 0.320 — 0.254 0.315 — 0.317 0.365 — 0.316 0.361 — 0.288 0.344 — 0.273 0.336 — 0.311 0.337 — 0.329 0.343 — 0.310 0.350 — 0.295 0.338 — 0.328 0.346
ETTh1: 0.390 0.418 — 0.411 0.434 — 0.395 0.420 — 0.400 0.424 — 0.394 0.419 — 0.412 0.426 — 0.404 0.417 — 0.417 0.419 — 0.480 0.439 — 0.591 0.468 — 0.588 0.466 — 0.473 0.443
ETTh2: 0.340 0.387 — 0.333 0.387 — 0.334 0.387 — 0.366 0.404 — 0.405 0.415 — 0.371 0.399 — 0.347 0.388 — 0.362 0.382 — 0.367 0.377 — 0.405 0.410 — 0.455 0.427 — 0.392 0.406
ECL: 0.169 0.265 — 0.169 0.265 — 0.166 0.262 — — — — — — Other baselines: 0.174

Zero-shot probabilistic forecasting on FEV

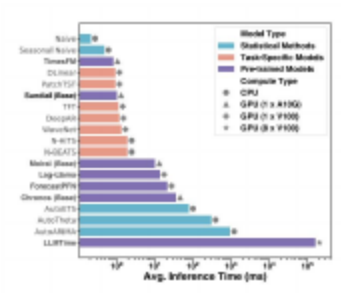
- We outperform most supervised/statistical baselines zero-shot.
- We are competitive among pre-trained models with strong MASE/WQL.
- We generate 20 samples to deliver calibrated intervals.



We evaluate zero-shot on FEV; lower MASE/WQL is better.

Inference speed: Milliseconds with just-in-time sampling

- We approach N-BEATS latency and are 35× faster than Chronos on FEV.
- We reduce steps via patch-wise and multi-patch generation.
- We cut memory/time with FlashAttention and KV cache.



We report average latency on FEV (log-scale x-axis) with annotated compute budgets.

Ablation: Training objectives (probabilistic quality)

- We achieve the best CRPS on GIFT-Eval and across datasets.
- We also improve MSE vs. diffusion and MSE objectives (see Slide "TSLib results").
- We confirm the advantage of generative distribution learning and calibration.

Table: Table 7. Zero-shot probabilistic forecasting with different training objectives. Averaged CRPS (lower is better).

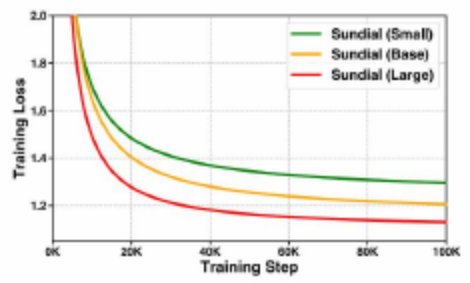
Objective	ETTh1↓	ETTh2↓	ETTm1↓	ETTm2↓	ECL↓	Weather↓	GIFT-Eval↓
TimeFlow Loss	0.0059	0.0037	0.0057	0.0029	0.0082	0.0021	0.5050
Diffusion Loss	0.0082	0.0053	0.0070	0.0039	0.0095	0.0032	0.5340
MSE Loss	0.0063	0.0040	0.0058	0.0032	0.0080	0.0023	0.6420

Engineering ablations: What makes it fast and stable?

- RoPE improves zero-shot; Pre-LN stabilizes convergence.
- Cut memory by 14.8%.
- Reduce inference time by 43.6%.

Scaling behavior: Larger models, lower loss

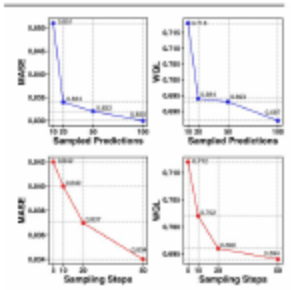
- We observe consistent capacity utilization in training curves.
- Large reduces converged loss by 15.38%.
- We benefit from scaling both data (TimeBench) and model size.



We plot TimeBench pre-training loss for different model sizes.

Test-time calibration: Accuracy–latency control

- We improve MASE and WQL by increasing samples or steps K without retraining.
- 1 second on CPU with 20 samples $\times$ 50 steps.
- We offer a flexible trade-off across deployment budgets.



We show FEV MASE and WQL trends as we vary sample count and sampling steps.

Limitations and considerations

- We currently pre-train on univariate series; multivariate/covariates are future work.
- We still find very high-frequency and ultra-long contexts challenging.
- We can further improve calibration via sampling design and post-processing.

Conclusion: What we deliver with Sundial

- We provide native, flexible generative forecasting for continuous time series.
- We avoid priors and mitigate mode collapse with TimeFlow Loss.
- We achieve state-of-the-art zero-shot accuracy with millisecond inference.
- We leverage TimeBench to study scaling laws and robust generalization.

Baselines and sources

- We compare to: Time-MoE [Shi et al., 2024b], Timer-XL [Wen et al., 2024], Moirai [Jain et al., 2024], Chronos [Rasul et al., 2024], TimesFM [Das et al., 2024], N-BEATS [Oreshkin et al., 2020].
- We evaluate on datasets: TSLib [Wu et al., 2022], FEV [Ansari et al., 2024], GIFT-Eval [Shi et al., 2024a].
- We follow author–year strings consistent with the paper's references.

Questions & discussion

- We appreciate your attention and feedback!
- We welcome questions and discussion.